

Example Complete stack

```
mern-demo/
├── mern-backend/
│   ├── server.js
│   ├── package.json
│   └── node_modules/
└── mern-frontend/
    ├── public/
    ├── src/
    │   ├── App.js
    │   └── index.js
    ├── package.json
    └── node_modules/
```

Express + MongoDB (server side)
Your backend entry file
Created by npm init
Installed dependencies (express, mongoose, cors)

React app (client side)
Static files (index.html, etc.)
Your React code
React entry point
Created by npx create-react-app
Installed React dependencies

npm start 3000
node server.js 5000

Server.js BACKEND

npx create-react-app .

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
```

```
const app = express();
app.use(cors());
```

```
// Connect to MongoDB
mongoose.connect('mongodb://localhost:27017/studentdb')
  .then(() => console.log('MongoDB Connected'))
  .catch(err => console.error(err));
```

```
// Define Student schema
const studentSchema = new mongoose.Schema({
  rollno: Number,
  name: String,
  course: String,
  semester: Number
});
```

```
const Student = mongoose.model('Student', studentSchema);
```

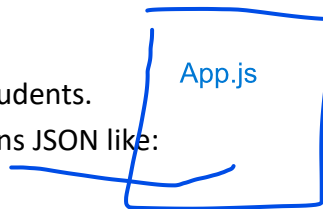
```
// Sample route: get all students
app.get('/students', async (req, res) => {
  const students = await Student.find();
  res.json(students);
});
```

```
// Start server
app.listen(5000, () => console.log('Server running on http://localhost:5000'));
```

MongoDB stores student data.

Express provides an API endpoint /students.

http://localhost:5000/students returns JSON like:



server.js



mongodb

```
[
  { "rollno": 1, "name": "Amit", "course": "BSc CS", "semester": 3 },
  { "rollno": 2, "name": "Priya", "course": "MBA", "semester": 1 }
]
```

App.js Frontend

```
import React, { useEffect, useState } from 'react';
```

```
function App() {
```

```
  const [students, setStudents] = useState([]);
```

```
  useEffect(() => {
```

```
    fetch('http://localhost:5000/students')
```

```
    .then(res => res.json())
```

```
    .then(data => setStudents(data));
```

```
  }, []);
```

```
  return (
```

```
    <div>
```

```
      <h1>Student List</h1>
```

```
      <ul>
```

```
        {students.map(s => (
```

```
          <li key={s.rollno}>
```

```
            {s.rollno} - {s.name} ({s.course}, Semester {s.semester})
```

```
          </li>
```

```
        )}}
```

```
      </ul>
```

```
    </div>
```

```
  );
```

```
}
```

```
export default App;
```

BACKEND setup and Testing

Setup: Backend Setup (Express + MongoDB)

1. Create a folder for backend, e.g. mern-backend.
2. Open it in VS Code.

npm init -y
npm install express mongoose cors

project backend structure is created

start the backend server

node server.js

paste this server.js

Testing: Run MongoDB and insert student records.

```
db.students.insertMany([
  { rollno: 1, name: "Amit", course: "BSc CS", semester: 3 },
  { rollno: 2, name: "Priya", course: "MBA", semester: 1 }
])
```

Start Express (node server.js).

Start React (npm start).

Open <http://localhost:3000> → Students list appears.

FRONT END Setup

Create mern-frontend folder

Execute command in this folder

npx create-react-app . (dot indicates current directory)

change to mern-frontend

replace code of App.js /src/App.js

npm start

Two separate projects:

- **mern-backend** → runs on Node.js with Express, talks to MongoDB.
- **mern-frontend** → runs on React, fetches data from backend.

Start commands:

- Backend: node server.js (runs on port 5000).
- Frontend: npm start (runs on port 3000).

Connection: React (<http://localhost:3000>) fetches data from Express (<http://localhost:5000/students>).